

Dippi: Architecture, Trust Boundaries, and Reproducible Measurement of a Testnet Spot Exchange

Technical Report and Measured CPU Benchmark Campaign

Arya Somu

Abstract. Dippi is the consumer identity for an experimental spot-exchange system whose source tree retains Vela protocol and package names. This report describes the implemented Rust central limit order book, signed-order API, custody contract, PostgreSQL ledger, deposit indexer, withdrawal workflow, reconciliation, recovery, and verification research. It separates source code, local tests, deployed behavior, and architectural intent. On a rented 16-vCPU AMD EPYC 9754 allocation, the baseline measured a median 1.245 million in-process operations/s for a validated synthetic mix. Its loopback signed-order HTTP p50 was 183 ms for one client and 493 ms for 32 clients; the latter had 1,920 HTTP 400 responses in 16,939 attempts. A focused test reproduced a cross-market balance-reservation error. A repaired source archive passed that regression and, on a second allocation of the same class, measured 1.185 million in-process operations/s while recording median repeat-level HTTP p50 of 27.60 ms for one client and 35.67 ms for 32 clients. All 132,907 HTTP attempts in those scenarios succeeded. Its five-minute workload achieved 145.05–165.83 successful operations/s and 1.02–1.30 GB end RSS, while 10,412 of 148,986 attempts failed without recorded status detail. These measurements do not include PostgreSQL financial commit, real network latency, custody settlement, or a production customer service level. Dippi remains a testnet research platform.

Report date: October 8, 2026. **Source baseline:** 940f3a48fe413fa83ff997fa3d6a770ac9488df8.
Follow-up source: archive SHA-256 in Appendix D.

1 Introduction and evidence rules

Dippi is a consumer brand; Vela remains the name of several Rust crates, Solidity contracts, HTTP fields, and historical documentation. Renaming these identifiers in prose would obscure what the source actually implements. The baseline starts at the remote main commit shown above. Its benchmark harness changes have digests in the run records. The follow-up uses an exact source archive identified by SHA-256 to identify the measured files independently of later repository revisions. The source repository is private as of this report; commit and path citations identify reviewable material for authorized readers, but they do not provide unrestricted public reproduction. A public replication package would require an explicit source-release decision. The report must be revised if either evaluated source or harness changes.

The word *implemented* means code exists and can be inspected. *Locally demonstrated* means a named test or rehearsal exercised it under controlled local dependencies. *Hosted demonstrated* requires a specific deployment, commit, contract, chain, timestamp, and captured result. *Production qualified* additionally requires the applicable audit, operations, legal, and go/no-go evidence. These categories are deliberately non-interchangeable. Architectural decision records (ADRs) specify intended invariants but do not by themselves prove implementation. A green CI check proves only the checks run on that revision. The source of truth for the financial-release boundary is the accepted ADR set and the M8 evidence index, read together with code and tests.[\[Dip26c,Dip26g\]](#)

This document is a technical report about a testnet implementation. It does not invite deposits of real funds, make a claim of public solvency, or assert that a proof system can enforce a customer withdrawal. The older Vela paper is historical context, not a source of current benchmark results.[\[Som26\]](#)

1.1 Reconciliation with the April 2026 paper

The earlier paper described a research architecture and measurements at a different revision. Its abstract should not be read as a current deployment or performance statement. Table 1 identifies the largest changes and qualification gaps; the sections below give the source-level detail.[\[Som26,Dip26e\]](#)

Earlier description	Current source-grounded interpretation
Merkle Patricia root committed to a chain	MptStore computes a sorted-map whole-state digest; a distinct sparse Merkle tree supports per-key proofs. This report does not identify a deployed chain commitment for either root.
Optimistic zero-knowledge challenges and a seven-day verifier/challenge settlement window	The default prover can return a skipped result, and no deployed path is established by this source review.
Self-custodial exchange characterization	The proposed first customer system uses an omnibus V3 vault and a PostgreSQL customer ledger. Withdrawal authority is mediated by off-chain policy and an EIP-712 signer.
Microsecond latency and 57,300 mixed operations/s	Historical figures use older fixtures and incompatible timing boundaries. New measurements are reported with their operation unit, status counts, source digest, and machine allocation.

Table 1. Older descriptions versus the reviewed testnet implementation. This is a source and evidence comparison, not a claim that the historical experiment never occurred.

1.2 System boundaries

The public brand site, Next.js frontend, Rust HTTP/WebSocket API, matching engine, PostgreSQL ledger, chain indexer, withdrawal worker, reconciliation worker, and Solidity vault form different operational boundaries. The core matching benchmark times only a subset of them. The public holding page is not evidence that a customer flow is deployed. The local PostgreSQL and Anvil harness is not independent Base Sepolia provider or hosted custody evidence.[\[Dip26f\]](#)

Boundary	Authority or function	Evidence limit
Matching engine	Deterministic order and fill proposals	In-memory state is not the customer money record.
PostgreSQL ledger	Customer ownership, reservations, journal and projections	Local transaction tests do not establish hosted resilience.
V3 custody vault	Holds allowlisted ETH/token assets; validates signed withdrawals	Source and Foundry tests do not establish a deployed, audited vault.
Indexer and workers	Observe finalized chain evidence and reconcile lifecycle state	Two local RPC URLs may share one Anvil process.
SMT, DA, proof and TEE modules	Research mechanisms for integrity and verification	No deployed on-chain challenge or validity enforcement is claimed.

Table 2. Authority and evidence boundaries at the reviewed revision.

2 Architecture and transaction flow

An authenticated order enters the Axum API, where validation and wallet-signature verification precede engine dispatch. The matching engine applies price-time priority in an in-memory order book and emits order, fill, balance, or error responses. In ledger-backed mode, the intended externally visible commit boundary is a serializable PostgreSQL transaction that records the financial effect set. The ADR requires acknowledgment and feeds to follow the durable commit, not a speculative in-memory transition.[\[Dip26c\]](#) The exact runtime route and rollback behavior must be evaluated at the source revision; the isolated engine microbenchmark bypasses this coordinator.

The exchange is not a blockchain consensus protocol. Even when a batch digest or state root is computed, that computation alone does not make matching decentralized. The operator controls admission, ordering, service availability, and retained data until independently enforceable mechanisms exist. The design explicitly separates the first, ledger-first consumer architecture from later trust-minimization research.[\[Dip26d\]](#)

2.1 Module map

The Cargo workspace includes `types`, `engine`, `state`, `api`, `committer`, `zkvm`, `tee`, `committee`, `perp`, `fix`, `ledger`, `indexer`, `withdrawal-worker`, `reconciliation-worker`, and `benches`. Solidity and the Next.js frontend are separate build surfaces. Appendix A records their role and current qualification boundary.

3 Matching and market-state machinery

3.1 Order book and numerical representation

The order book stores bid and ask price levels in ordered `BTreeMap` collections and each level's resting orders in a FIFO `VecDeque`. An order-ID index maps to side and price for cancellation; a separate customer/client-order-ID index rejects duplicate caller references. A better bid or ask has price priority; an earlier resting order at a price has time priority. The book has a configured maximum resting-order count and returns an explicit full-book error. Iceberg orders carry hidden reserve as well as displayed quantity: matching-side depth includes the full remaining matchable size, whereas public depth and feeds use visible quantity. These definitions matter when interpreting a “book depth” workload.[\[Dip26e\]](#)

Prices and quantities are represented in fixed-point integer units, each with a scale of 10^8 in the engine. Conversion to an asset's configured base units at the ledger boundary multiplies or divides by the appropriate power of ten; division must be exact or the operation is rejected. For six-decimal USDC, an engine quantity must be divisible by 100. Floating-point figures in documentation must never be mistaken for financial storage values.[\[Dip26e\]](#)

The hot path supports good-till-canceled, immediate-or-cancel, fill-or-kill, and post-only orders. For a resting limit order, the engine validates market, ticks, balance, credit, nonce, and advanced controls before changing book state. A taker order consumes eligible resting liquidity. A canceled remainder for immediate-or-cancel is different from a full fill. Fill-or-kill must either execute its required quantity atomically or leave no partial financial effects. Post-only must reject when it would cross. Per-order self-trade prevention has skip, cancel-taker, cancel-maker, cancel-both, and decrement/cancel modes; the latter has a documented v1 limitation when the taker is smaller than the maker. Tests and benchmark workloads must inspect response variants and final book state, not merely count invocations of `engine.process`.[\[Dip26e\]](#)

3.2 Mutation isolation, nonces, and credit

The engine uses a delta buffer to stage state mutations, allowing a failed operation to discard its proposed changes. A rolling nonce window permits limited concurrent, non-sequential order submission while rejecting replay. Client order identifiers provide lookup and cancellation by a caller-supplied stable reference. Market-maker credit logic bounds aggregate quoting against configured collateral and can cancel resting orders to restore the credit constraint. Maker/taker fee parameters and maker rebates affect both matching responses and, when configured, journal effects. The toxicity scorer uses fixed-size order-flow imbalance history and fill context to attach an adverse-selection score. These mechanisms are implemented in the engine source, but their measured overhead is workload-dependent.[\[Dip26e\]](#)

3.3 Batching and market shards

The batch dispatcher opens a window on the first arrival and fires at the earlier of its configured duration or maximum count (defaults: 500 μ s and 256 requests). Responses return through per-request one-shot channels. The market-shard path groups work by market and joins shard futures. The API installs a durable mutation committer even without PostgreSQL: its dispatch path locks committed state, clones the user state into a private proposal, executes the batch, flushes the local mutation WAL, and then publishes the proposal and responses. The measured baseline also clones every market shard on every batch. The follow-up source clones only shards touched by that batch and swaps successful proposals into committed state, but still clones the full user state and takes a full balance/metadata snapshot. With a configured ledger it additionally constructs and commits

financial effects. Thus the loopback HTTP fixture’s 11,000 funded accounts can materially affect clone cost on every batch, while the isolated engine benchmark does not exercise that path. Throughput and latency also depend on arrival rate, batch occupancy, queueing, and financial mode. A single-threaded `MatchingEngine.process` benchmark does not establish the throughput of the sharded path, the dispatcher, or the durable ledger commit.[\[Dip26e,Dip26h\]](#)

The measured baseline’s `BatchMetrics::record_batch` appended every batch size to a vector while the inspected API metrics read took a non-draining snapshot. The follow-up source replaces that vector with fixed-size cumulative counters. Five-minute resident-set measurements still cannot establish indefinite memory stability or isolate other retained state.[\[Dip26e\]](#)

4 Authoritative financial architecture

4.1 Journal and commit point

The accepted financial model uses PostgreSQL as the system of record. Each financial command has an idempotency key and produces balanced integer-denominated legs. A serializable transaction locks affected balances in stable order, verifies available funds, writes immutable journal and related records, updates projections, and commits before acknowledging financial success. Recovery and customer statements are projections of that journal. The matching engine’s balance map and write-ahead log are execution aids, not independent financial authority.[\[Dip26c\]](#)

The `ledger` crate contains schema migration, posting, idempotency, projection, administrative corrections, reconciliation, and recovery watermarks. The API’s ledger coordinator converts engine amounts to ledger asset units, constructs effect legs, and restores engine financial state from committed data. A full benchmark of an order in this mode must include reservation, serializable commit, outbox effects, and acknowledgment, with the database configuration and durability setting recorded. The current matching microbenchmarks do not include these costs.

For a journal transaction t and asset a , every set of legs must satisfy

$$\sum_{\ell \in t: \text{asset}(\ell)=a} \text{amount}(\ell) = 0. \quad (1)$$

The materialized account balance is a projection of the immutable leg sequence, not an independently editable source. The initial schema constrains customer accounts to nonnegative balances and identifies available, locked, pending, and withdrawal-reserved customer states; custody-vault and custody-withdrawn counter-accounts; and platform fee, rebate, and operational-adjustment accounts. The transaction also writes an outbox event. A reused idempotency key with a different request hash is rejected rather than silently treated as a replay.[\[Dip26e\]](#)

In ledger-backed mode, the dispatcher builds a private copy of committed engine state, executes the batch, computes net available/locked and fee effects, verifies that each asset balances, converts units exactly, and posts one serializable transaction. Only after that commit may it publish the proposed state and release responses. On a failed validation or write, the proposal is discarded. Startup replays journal legs under a consistent snapshot, compares them with every account projection, and replaces cached customer financial balances; a mismatch aborts startup. A singleton PostgreSQL leadership lease fences concurrent matcher generations. Development and `testnet-demo` can run without this ledger, so a benchmark of those modes must not be relabeled as a durable financial transaction.[\[Dip26c,Dip26h\]](#)

4.2 Custody contract

The `V3 VelaSettlement` source is an immutable omnibus vault with no on-chain per-customer trading balances. It accepts native ETH and allowlisted token deposits, records the measured token receipt, and emits deposit evidence. A withdrawal is executable by a relayer only with a policy-authorized EIP-712 signature binding request ID, destination, asset, amount, nonce, deadline, policy hash, and approver hash. Consumed request IDs and nonces prevent replay. The source separates deposit and withdrawal pauses, constrains operator rotation, checks the L2 sequencer condition, and limits migration through owner, guardian, time-delay, operator-signature, and liability-snapshot conditions. None of these contract checks alone proves that the off-chain ledger is solvent.[\[Dip26e\]](#)

The older Sepolia contract described in the original paper is a different custody model. In particular, the V3 vault has no customer self-service emergency exit. A report that combines the old emergency-exit claim with V3’s omnibus accounting would describe no actual deployed architecture. The source and ADR must be identified separately from any old testnet address.[Dip26c,Som26]

4.3 Deposit admission

The Base deposit indexer validates a deployment manifest and exact runtime code before accepting an event source. Its locally tested path observes ETH and six-decimal USDC self-deposits, checks the canonical event identity, compares independent provider observations, waits for the configured finalized state, and journals an idempotent credit to the indexed wallet owner. A provider contradiction or finalized-history conflict must quarantine or halt rather than credit optimistically. A local Anvil integration test proves wiring and controlled failure paths; it cannot prove that two named local URLs are independent RPC providers.[Dip26f]

4.4 Withdrawal lifecycle

A customer withdrawal begins with authenticated intent and ledger reservation. Policy approval, a bounded nonce and fee decision, exact signer preflight, signed transaction preparation, broadcast, chain observation, replacement lineage, finality, and either completion or exactly-once release are separate transitions. The implementation includes durable records for several of these boundaries and a deliberately disabled production broadcaster. The M8 evidence package still lists a real signer, deployed V3 transactions, two independent RPCs, and finality plus reconciliation proof as open. It would be false to infer a working public withdrawal service solely from source-level state machines.[Dip26g]

4.5 Reconciliation and recovery

Reconciliation is asset scoped. At an exact finalized block b , two independent providers must agree on chain ID, block hash, timestamp, vault bytecode, native balance, and registered-token balances. Let $C_a(b)$ be observed vault custody for asset a and $L_a(s)$ the ledger’s non-custody liability projection in the reconciliation transaction snapshot s . The safety condition checked for that observation is $C_a(b) \geq L_a(s)$; the worker independently checks liability against the negated custody counter-accounts. Provider disagreement, missing projections, negative customer balances, or a deficit makes the result indeterminate or deficient and halts the asset. A surplus is recorded as surplus, possibly from a finalized but not yet credited deposit, rather than declared balanced. An operator must resolve a halt deliberately.[Dip26e]

Administrative corrections require incident and evidence references, an initiator, a distinct approver, an approval timestamp, and additive balanced legs rather than destructive journal edits. Local backup, signed-bundle, WAL, and named-point PostgreSQL restore rehearsals are useful evidence. They do not establish managed replication, off-host recovery, paging, or a zero-loss production recovery objective. This benchmark campaign does not treat a successful local restore as a throughput figure or as production disaster-recovery proof.[Dip26c,Dip26g]

5 Integrity, confidentiality, and experimental modules

The **state** crate has two different root structures. Despite its name, **MptStore** is an ordered key/value map whose root is a Keccak-256 hash of the length-prefixed concatenation of every sorted key and value. It is a deterministic whole-state digest, not a Merkle Patricia trie with compact per-key proofs. The separate **SmtStore** is a depth-32 sparse Merkle tree keyed by the first 32 bits of a key hash. It groups full-hash collisions in a sorted leaf bucket and supports inclusion and non-inclusion proofs with 32 sibling hashes. A statement about an “MPT proof” cannot be inferred from the **MptStore** name.[Dip26e]

StateManager keeps both structures, but the inspected disk-load path restores the flat map and cache without rebuilding the sparse tree. Its snapshot persistence logs an error and still returns a batch root if a local write fails. These are concrete recovery and durability limits of the research commitment path. The **committer** packages a sequence, root, snapshot, and requests for a data-availability client; its local backend writes a content-hashed blob to disk. A local receipt is evidence

of local storage, not a public availability guarantee or a commitment to the authoritative PostgreSQL liability projection.[\[Dip26e\]](#)

The `zkvm` crate can replay a state transition from a snapshot and compare roots, but its default prover returns a skipped proof with no proof bytes; an SP1 path depends on external proving infrastructure. The `tee` crate exposes an attestation abstraction and placeholder implementation. The `committee` crate explores threshold decryption and BLS cryptography. These modules do not by themselves provide a deployed fraud-proof contract, hardware attestation, or a censorship-resistant L1 inbox. The committed statement, public inputs, data availability, verifier, and challenge or settlement path must each be named before a third party can rely on a proof claim.[\[Dip26e,Dip26g\]](#)

Perpetual margin and funding, borrow/lend, portfolio margin, FIX connectivity, RFQ, strategies, agent capabilities, subaccounts, and related API modules exist at varying levels of integration. They are outside the proposed first real-money ETH/USDC spot scope. The report documents their interfaces and test status as experimental extensions, not as customer guarantees.[\[Dip26d\]](#)

6 API, authentication, and customer surfaces

The API offers HTTP order and account routes, WebSocket market and private account feeds, and a FIX adapter. Wallet-signed orders bind market, side, price, quantity, and nonce; version 2 signatures additionally bind advanced execution controls. Session challenges and account-bound bearer tokens protect customer-data reads. WebSocket feeds carry sequence numbers that allow a client to detect gaps, but sequence numbers do not replace durable replay or a committed financial acknowledgment. Rate limits, capability scopes, and revocation are meaningful only in the runtime and mode where their backing stores are active.[\[Dip26e\]](#)

The frontend, TypeScript and Python SDKs, API schema, and Mintlify documentation describe access paths. The current public Dippi surface is a holding page. The complete customer lifecycle, including mobile wallet recovery, receipts, operational guidance, and support, remains a separate launch qualification question. A technical report should never promote a route or UI module to “live product” status based on its presence in the repository alone.[\[Dip26f,Dip26g\]](#)

7 Benchmark design and reproducibility

7.1 Questions and measurement boundaries

The campaign asks four different questions: (1) how fast the isolated matcher processes a specified valid request mix, (2) how performance varies by order type, fill ratio, book depth, and batch size in exploratory fixtures, (3) what a successful signed-order HTTP round trip costs on loopback, and (4) how successful HTTP throughput and memory behave over five minutes. The first two do not include network or ledger commit. The HTTP suites use a locally hosted API, temporary local WAL and DA directories, and pre-funded synthetic users. They include the configured local durability path but no PostgreSQL financial commit or remote network hop. The concurrent HTTP latency scenario has 32 clients with a 50 ms think interval after each request; this bounds resting orders per synthetic account and defines its offered load. Its three scenarios run in sequence against one server and one growing set of order books. Their differences cannot be attributed solely to concurrency. The burst spawns 1,000 client tasks without a synchronized network start. The sustained suite runs the client agents and loopback server in the same process, so its resident-set samples include both sides and cannot be attributed to the server alone. The HTTP benchmark runner uses a 5,000 ms engine-dispatch deadline and a new temporary admin token for each suite. No suite measures an audited, deployed Base custody transaction. These omissions and settings govern every result table.

7.2 Workload validity

The standard engine run seeds ten markets, 50 market makers, one taker, and 2,000 resting orders. It prepares 4,900 cancel/repost pairs and 200 taker IOC orders. For this revision of the benchmark harness, each cancel targets the current resting order ID, the validation pass checks cancellation and repost responses, and the taker phase must produce fills. Earlier streams recycled only initial order IDs and could time rejected cancellations. The HTTP latency fixture funds 11,000 synthetic keys with

quote and base assets for eleven markets; the sustained fixture funds 10,000 keys. Key generation, account funding, and server construction occur before the timed windows. The HTTP latency harness records each attempt and status, includes only successful responses in latency percentiles, and reports failures separately. The sustained harness also reports attempts, successes, and failures separately. Historical numbers obtained before these changes are not directly comparable to new figures.[Dip26e]

The older `benches/benches/matching.rs` suite uses a different ten-market, 10,000-resting-order fixture. Several stateful groups record a new order ID under a placeholder owner or market rather than the returned order’s actual owner and market, and the random-cancel path does not assert cancellation success. A later cancellation can therefore time an error response. The engine currently ignores deposit event hashes, so repeated hashes in this synthetic fixture do not test chain-admission idempotency either. The campaign retains this suite’s raw outputs for diagnostic coverage but excludes its figures from validated performance headlines until its request outcomes and bookkeeping are corrected.[Dip26e]

Suite	Work unit and traffic	Timed boundary and validity rule
Engine	Ten-market 9,800 cancel/repost operations plus 200 IOC takers; isolated order variants.	<code>MatchingEngine::process</code> ; validate response and actual fills. Criterion iterations are aggregate timing estimates.
Matching	Posts, cancels, full loops, batch sizes, fill ratios, concurrent takers, bursts, and book-depth sweeps.	In-process benchmark-specific calls; groups have different operation denominators and cannot be pooled.
HTTP latency	One sequential client, 32 clients with 50 ms think time, then a 1,000-request burst using disjoint synthetic keys.	Signed <code>POST /orders</code> on loopback through complete response body; all statuses recorded, successful responses alone enter latency statistics.
Sustained	Five maker agents for 300 seconds; branch weights are 60% cancel/repost, 20% new limits, 15% aggressive IOC, 5% public market reads. An empty resting-order queue turns a cancel branch into a new post.	HTTP response-header latency for successes; attempted, successful and failed requests and five-second throughput buckets are separate outputs.

Table 3. Benchmark suites and their distinct measurement boundaries.

7.3 Run record

Each run saves the source commit, benchmark-source diff digest, Rust version, operating system, CPU model, logical CPU count, memory, benchmark profile, repetition number, stdout, Criterion artifacts, HTTP attempt latencies and statuses, and sustained five-second throughput buckets. A cloud CPU pod’s host model and rented allocation are recorded separately. The timed matching path has no GPU kernel, so a GPU rental would not accelerate this measurement. Measurements on the Apple M3 and on a cloud x86 CPU remain separate hardware strata. The result tables state attempt and success counts, successful-response percentiles, error counts, and variation across repetitions. No cross-machine speedup is presented as an algorithmic improvement.

7.4 Machine, source, and archive identity

The baseline cloud campaign ran on a Runpod `cpu3c` allocation with 16 available logical CPUs and a 32GB memory cgroup. The host reported an AMD EPYC 9754 128-Core Processor and 256 host logical CPUs; those host totals are *not* the rented allocation. The container ran Linux 5.15.0-94-generic x86_64, Rust 1.99.0, Cargo 1.99.0, and the Cargo benchmark profile with fat LTO and one codegen unit. No GPU was used. The machine was shared infrastructure, so neighboring-load and clock-state variation remain possible. Each baseline suite had three repetitions; HTTP and sustained ran together in each repetition. The follow-up used a separate pod of the same CPU allocation

class and separate HTTP, sustained, engine, and legacy matching invocations. The run IDs, source identities, and archive digests are in Appendix D. The baseline archive was copied to local storage and its SHA-256 matched the remote file before that pod was stopped.[Dip26a,Dip26b]

7.5 Validated isolated engine workload

Criterion reports a mean for a 10,000-request iteration, not individual-request latency samples. The validated stream consists of 9,800 cancel/repost operations and 200 taker IOC orders after setup, with the required outcomes checked before timing. Table 4 divides each iteration mean by 10,000 to express an amortized operation cost. The median across the three repeat means is 1,244,755 operations/s, with a repeat range of 1,212,524–1,258,417. These are in-process `MatchingEngine::process` rates; there is no HTTP, dispatcher, financial journal, chain, or remote client in this timed unit.[Dip26a]

	Repeat	Criterion mean/10k (ms)	Amortized ns/op	Operations/s
	1	7.946	794.6	1,258,417
	2	8.034	803.4	1,244,755
	3	8.247	824.7	1,212,524

Table 4. Validated synthetic in-process matching mix. Each row is one Criterion mean, not an operation-level percentile.

The exploratory legacy `matching` run produced 30 Criterion estimate artifacts per repetition. Its stateful ownership and cancellation bookkeeping do not validate successful work across all groups, so those artifacts are retained in the private archive but are excluded from performance conclusions. The historical 57,300 mixed operations/s figure is not a comparable prior observation for this corrected stream.

7.6 Signed-order loopback HTTP

Table 5 shows every cloud repetition. Attempt counts include all returned statuses; p50 and p99 use only successful complete-body responses. The single-client scenario had 1,902/1,902 successes, with median repeat-level p50/p99 of 183.24/288.89 ms. The 32-client scenario had 15,019/16,939 successes and 1,920 HTTP 400 responses, with median repeat-level p50/p99 of 493.10/643.80 ms. Its failure fraction varied from 3.79% to 21.46% across repetitions. The 1,000-task burst had 3,000/3,000 successes; median repeat-level p50/p99 was 2,125.51/3,287.29 ms. These median-of-repetition percentiles are descriptive, not pooled confidence intervals. All HTTP 400s recorded in this campaign carried an insufficient-balance response, but their individual causes have not each been traced.[Dip26a]

Scenario	Repeat	Successful	HTTP 400	p50 (ms)	p99 (ms)
One client	1	623/623	0	182.34	288.89
	2	637/637	0	184.10	299.92
	3	642/642	0	183.24	274.59
32 clients	1	4,867/5,323	456	490.83	592.45
	2	4,549/5,792	1,243	529.79	1,441.34
	3	5,603/5,824	221	493.10	643.80
1,000-task burst	1	1,000/1,000	0	2,125.51	3,282.83
	2	1,000/1,000	0	2,191.98	3,432.08
	3	1,000/1,000	0	2,048.68	3,287.29

Table 5. Loopback signed-order HTTP, complete-body successful-response latencies. Scenario order is fixed, so later scenarios inherit a changed server state.

This path includes signature validation, Axum routing, dispatcher queueing, private state proposal, local WAL durability, and full HTTP response receipt. It runs without PostgreSQL or a network hop.

The benchmark process holds the client and server, and the 11,000 synthetic accounts are cloned as part of each proposal. The observed HTTP time must not be attributed to matching alone. Likewise, the 32-client rejection fraction makes successful-response p99 an incomplete user-experience statistic; failure counts must travel with it. Scenario order, growing books, synthetic funding, and uncontrolled burst launch prevent a causal concurrency speedup interpretation.

7.7 Five-minute sustained workload

The sustained process used five agents over 300 seconds per repetition. Successful operations include public market-data reads as well as order actions. It completed 17,892 of 18,936 HTTP attempts across all repetitions, with 1,044 failures (5.51%). Successful rates were 19.76–19.96 operations/s. The five-second buckets summed exactly to each run’s success count and none of the 180 buckets had zero successful throughput. This shows short-horizon progress under the offered synthetic load, not a maximum sustainable capacity or an indefinite stability guarantee.[\[Dip26a\]](#)

Repeat	Successful	Failed	Success/s	p50 (ms)	p99 (ms)	Fills	RSS MB 0/150/300s
1	5,988/6,338	350	19.96	232.63	498.70	446	496/9,537/9,943
2	5,928/6,275	347	19.76	237.37	482.98	439	497/10,733/11,349
3	5,976/6,323	347	19.92	233.74	460.28	444	497/9,722/10,927

Table 6. Five-minute sustained loopback HTTP. Percentiles are successful response-header latency; RSS includes the same-process clients and server.

The observed resident set rose from about 496–497 MB at start to 9.94–11.35 GB at five minutes. This is substantial process memory growth in the measured fixture. Three coarse samples cannot separate retained book/account state, queued client data, allocator behavior, and a leak, or establish a long-run plateau. Five-second bucket minima were 7.8, 13.6, and 6.6 successful operations/s; maxima were 21.8, 22.0, and 22.2, respectively. Full buckets and raw successful latency samples are archived. The 1,920 32-client HTTP 400s and sustained 1,044 failures, together with the independently reproduced balance defect below, disqualify a service-level or financial-correctness claim from this campaign.

8 Reproduced cross-market balance defect

A focused local regression case at the measured source revision exposed an incorrect cross-market reservation transition. Two synthetic users each start with 1,000 units of USDC in the engine’s fixed-point scale. Alice posts a 100-unit bid in ETH–USDC, then a 100-unit bid in BTC–USDC. Bob then posts a separate 100-unit bid in ETH–USDC. All three posts return `OrderPosted`. Alice’s available/locked USDC should remain 800/200 after Bob’s unrelated order. Instead, her state moves from 800/200 back to 900/100. The second market’s reservation has been lost from the shared user state while its order can remain in a market book. The diagnostic test fails on the available-balance assertion (observed $900 \cdot 10^8$, expected $800 \cdot 10^8$).[\[Dip26e\]](#)

Source inspection identifies a mechanism consistent with this reproduction: a shard engine retains a full balance map from prior batches; each batch injects a fresh shared snapshot for request evaluation, but the merge compares every entry in the shard’s final map against the fresh snapshot, including entries that shard did not touch in the current batch. A stale entry can therefore generate a spurious delta that reverses another market’s earlier reservation. The HTTP workload’s rejected second orders were separately observed with zero available balance and an `InsufficientBalance` engine response. The regression proves the cross-market state transition error; it does not establish that every HTTP rejection has this sole cause.[\[Dip26e\]](#)

This was a financial correctness blocker in the measured baseline. The follow-up source limits shard balance and metadata maps to keys touched in the current batch and reads untouched keys from the shared snapshot. The reproduced sequence now leaves Alice at 800/200, and added tests cover duplicate nonces across markets and cancellation from a snapshot balance. The engine test suite

passes on the follow-up source. These tests address the named reproduction but do not constitute a full financial audit or a hosted PostgreSQL correctness proof. The baseline rejection counts remain part of its original record.[\[Dip26e\]](#)

9 Follow-up repair and comparative CPU campaign

The follow-up campaign tests an exact source archive derived from the baseline commit plus the repair and optimization worktree. Its SHA-256 digest, rather than the baseline commit alone, identifies the executable source. It used a second Runpod `cpu3c` allocation with the same 16-vCPU and 32 GB rental class and AMD EPYC 9754 host model. The baseline and follow-up differ in source and pod instance, so a latency difference is an observed comparison under the stated fixtures, not a controlled estimate of one code change’s isolated effect. Neither campaign uses a GPU.

The patch makes shard balance and metadata maps batch-local, reads untouched keys from the shared snapshot, tracks accepted nonces across markets within one batch, and limits durable proposals to touched market shards. A successful proposal is moved into committed state rather than copied a second time. Batch-size metrics use constant-space counters, and the HTTP order handler computes the toxicity tier once. The financial commit boundary remains intact: the API still acknowledges after the configured local WAL commit or, in ledger-backed mode, the required PostgreSQL transaction. The follow-up HTTP fixture again uses local WAL without PostgreSQL.[\[Dip26e\]](#)

The focused cross-market regression failed on the baseline and passes on the follow-up source. The complete `engine` test suite, 181 API library tests, and `cargo check -p api` passed locally on source matching the benchmark archive. The API library test run completed after generated build artifacts were cleaned to restore workstation disk space. These checks do not exercise the hosted PostgreSQL financial commit, deployed custody, or an external network, and they are separate from the benchmark observations.

9.1 Validated in-process matcher comparison

The same validated 10,000-operation synthetic stream ran three times on the follow-up allocation. Its median repeat-level rate was 1,185,063 operations/s, with a repeat range of 1,162,466–1,226,342; the baseline median was 1,244,755 operations/s. The follow-up median is 4.8% lower as a descriptive comparison across different pods. These changes target durable batch preparation and state publication rather than the isolated `MatchingEngine::process` hot loop, and this small cross-pod sample does not establish a causal matcher regression. The operation unit and excluded stages are unchanged from Table 4.[\[Dip26a,Dip26b\]](#)

Campaign	Repeat	Mean ns/operation	Operations/s
Baseline	1	794.65	1,258,417
Baseline	2	803.37	1,244,755
Baseline	3	824.73	1,212,524
Follow-up	1	815.43	1,226,342
Follow-up	2	843.84	1,185,063
Follow-up	3	860.24	1,162,466

Table 7. Validated in-process synthetic matching stream. Criterion iteration means are divided by 10,000; rows are separate repetitions, not individual-request latency percentiles.

9.2 Signed-order HTTP comparison

Three follow-up HTTP repetitions used the same 11,000-account signed-order fixture, 30-second warmup, 120-second measurement windows, 32-client think interval, 1,000-task burst, and 5,000 ms dispatch deadline as the baseline. They ran from an extracted source archive in two runner invocations; every run’s metadata records the same source digest. Table 8 reports median repeat-level successful complete-body latency, with every attempt retained in the denominator. The follow-up

observed zero failed attempts across 132,907 requests, compared with 1,920 failed attempts in the baseline’s 32-client scenario. The follow-up p50 was 27.60 ms for one client, 35.67 ms for 32 clients, and 339.63 ms for the burst. The baseline and follow-up hardware class matched, but these are different pod instances and the results cannot isolate the effect of each code change. Three repetitions do not establish a service-level bound.[\[Dip26a,Dip26b\]](#)

Campaign	Scenario	Successful/attempted	p50 (ms)	p99 (ms)
Baseline	One client	1,902/1,902	183.24	288.89
Follow-up	One client	11,060/11,060	27.60	65.33
Baseline	32 clients	15,019/16,939	493.10	643.80
Follow-up	32 clients	118,847/118,847	35.67	115.78
Baseline	Burst of 1,000	3,000/3,000	2,125.51	3,287.29
Follow-up	Burst of 1,000	3,000/3,000	339.63	412.63

Table 8. Loopback signed-order HTTP. Percentiles are medians of per-repetition percentiles among successful complete-body responses; they are not pooled confidence intervals.

9.3 Five-minute sustained HTTP comparison

The follow-up sustained workload used the same five-agent, 300-second fixture in three fresh processes. Table 9 reports every attempt and successful response, including public market-data reads. It recorded 138,574 successful operations in 148,986 attempts, or 145.05–165.83 successes/s across repetitions. There were 10,412 failures (6.99% of attempts). The harness records aggregate failures but not their HTTP status or response bodies, so their cause is unclassified. Its local queue may include an order ID after that order has filled, making later cancellation fail, but this code-level possibility is not evidence that every failure has that cause. No service-level or error-free sustained-throughput claim follows from these runs.[\[Dip26b\]](#)

Repeat	Success/attempt	Failed	Success/s	p50 (ms)	p99 (ms)	Fills	RSS MB 0/150/300s
1	45,312/48,704	3,392	151.04	26.55	81.85	4,327	93/1,177/1,177
2	43,514/46,771	3,257	145.05	28.23	80.01	4,161	93/932/1,017
3	49,748/53,511	3,763	165.83	24.69	67.41	4,815	93/1,289/1,301

Table 9. Follow-up five-minute loopback HTTP. Latencies end at successful response headers; RSS includes clients and server in one process.

All 180 five-second buckets had successful throughput, and each run’s bucket sum equaled its success count. End RSS was 1.02–1.30 GB, compared with 9.94–11.35 GB in the baseline. The baseline delivered 19.76–19.96 successful operations/s under its offered load and had 1,044 failures in 18,936 attempts. These observations show a large change in measured throughput and memory for this fixture, alongside a persistent non-success fraction. The follow-up’s higher arrival and fill counts, different pod instance, and three coarse RSS samples rule out attributing the entire difference to a single patch or claiming a stable long-run memory plateau.[\[Dip26a,Dip26b\]](#)

9.4 Remaining latency and capacity work

The follow-up changes reduce several avoidable copies but do not make the HTTP path an ultra-low-latency system. The following are source-visible costs or measurement gaps, not measured percentages of the observed p50 or p99.[\[Dip26e,Dip26b\]](#)

1. The durable batch path still clones the entire `UserState` and takes full balance and metadata snapshots. Key-scoped proposals or structural sharing would need to preserve rollback and the cross-market reservation invariant before replacing these copies.

2. The dispatcher defaults to a 500 μ s collection window and serial batch admission. Queue residence, occupied batch sizes, and contention need distributions under controlled arrival rates before changing the window or sharding policy.
3. The measured HTTP path verifies signatures, parses and serializes request bodies, and flushes a local mutation WAL before acknowledging. Per-stage traces and WAL size/fsync distributions are needed to rank these costs. Removing durability from the acknowledgment boundary would change the safety contract, not merely tune latency.
4. The ledger-backed path additionally constructs balanced financial effects and commits a serializable PostgreSQL transaction. This campaign did not run that mode, so no full financial-commit latency or maximum customer order rate is established. Hosted database behavior, conflicts, failover, and recovery need separate tests.
5. The HTTP client and server share one machine and loopback interface. External network RTT, TLS termination, ingress, WebSocket fanout, and realistic customer geography are absent. Their tail behavior cannot be inferred from loopback percentiles.
6. The sustained harness must record failure status and reason, distinguish invalid fixture actions from server errors, and report offered load as well as completed load. Longer steady-state runs and a memory profile are needed to establish a capacity knee and retention behavior.

Any next speed claim should report the same source identity, completed and failed counts, latency boundaries, offered load, database mode, and memory series. Improvements in the in-process matcher alone cannot certify this end-to-end path.

10 Threat model, limitations, and release gates

The operator can censor or reorder requests, withhold retained data, become unavailable, or misconfigure workers. A signed customer order prevents unauthorized order creation by a party without the key, but does not guarantee fair ordering or fulfillment. A custody vault can enforce withdrawal signature and pause rules, but cannot see off-chain beneficial ownership unless the ledger and policy process are correct. RPC disagreement, chain reorganization, bridge/token behavior, stale prices, database failure, relayer uncertainty, and signer compromise cross different trust boundaries. Each requires a defined fail-closed or recovery path and independent evidence.

Threat or failure	Implemented boundary	Residual qualification problem
Forged or replayed order	Wallet signature, bound nonce and execution controls, session and capability checks.	Compromised signing key and unsafe client recovery remain outside server validation.
Operator censorship	Ordering and DA research modules, a forced-inclusion queue.	No deployed censorship-resistant L1 inbox or guaranteed public data availability.
RPC disagreement or reorg	Manifest/code pinning, canonical event identity, finality wait, provider comparison and asset halts.	Local test URLs are not independent hosted providers; contradictory real-chain evidence needs rehearsal.
Ledger or writer failure	Balanced serializable journal, idempotency, private proposal, leadership generation, startup replay.	Hosted PostgreSQL durability, failover, restore, and incident response remain unproven.
Relayer or signer failure	Policy decision, exact signer preflight, nonce reservation, attempt lineage and terminal-state checks.	Production signer, real transaction evidence, and two-provider finality proof remain open.
Vault defect or insolvency	Immutable V3 rules, pauses, custody-to-liability reconciliation and asset halt.	No external audit or live hosted solvency record is claimed.

Table 10. Code-level controls and remaining evidence boundaries.

The current local harness uses synthetic balances and deterministic local chain fixtures. The M8 index still requires deployed V3 evidence, independent RPCs, external security review, production signer and database operations, incident rehearsal, legal approval, and a written go/no-go decision.

Until then, the correct product label is testnet research. Performance numbers from a rented CPU cannot close any of these gates.[Dip26g,Dip26d]

11 Conclusion

Dippi combines an in-memory matching research engine with a newer ledger-first custody design. The baseline synthetic matcher reaches a median 1.245 million in-process operations/s, but this unit omits HTTP and financial commit. Its loopback HTTP scenario recorded 1,920 rejections under 32 clients, its five-minute process reached roughly 10–11 GB RSS, and a local regression reproduced a cross-market reservation error. The follow-up source repairs the reproduced transition and passes the focused, engine-wide, and API library tests. Its isolated matcher median was 1.185 million operations/s on a second pod of the same CPU class, while loopback HTTP p50 fell to 27.60 ms for one client and 35.67 ms for 32 clients, with no failed attempts in the three HTTP scenarios. Its five-minute fixture completed 145.05–165.83 successful operations/s with 1.02–1.30 GB end RSS, while 6.99% of attempts still failed for reasons the harness did not classify. The measurements characterize two exact source states under synthetic local fixtures; they do not establish a production capacity, financial correctness, or service level. PostgreSQL financial commit, chain admission and settlement, and operational recovery still require separate measurements and independent evidence. Dippi remains testnet research.

A Repository module inventory

Module	Primary role	Qualification boundary
types	Fixed-point domain structures, requests, responses, Shared source types. markets, balances, nonces.	
engine	Order books, matching, delta buffer, credit, fees, tox- Isolated engine tests and icity, batching, shards.	benches.
state	Sparse Merkle tree, keys, caches, snapshots and state Local integrity implementa- management.	tion.
api	HTTP/WebSocket server, auth, feed routing, ledger Route-specific and mode- coordinator, runtime guards and experimental end-specific. points.	
committer	Batch construction, local DA, forced-inclusion No deployed delayed inbox. queue.	
zkvm	Replay and optional prover/verifier backends.	Placeholder by default.
tee	Attestation abstraction.	Hardware deployment un- proven.
committee	Threshold-decryption research.	Deployment and transport qualification open.
perp	Margin and funding calculations.	Outside first spot release.
fix	FIX 4.4 session handling.	Integration scope is partial.
ledger	PostgreSQL journal, balances, reservations, correc- Local financial tests; hosted tion and reconciliation records.	resilience open.
indexer	Base event admission, finality, provider comparison Local chain proof; inde- and deposit credit.	pendent hosted RPC proof open.
withdrawal- worker	Observation, lineage, finality and broadcast bound- Production broadcaster dis- aries.	abled.
reconciliation- worker	Custody-to-liability comparison and halt decisions.	Local proof; hosted alert/recovery proof open.
contracts	V3 omnibus vault and experimental ancillary Solidity Source/Foundry evidence, contracts.	no external audit.
frontend	Dippi public site and experimental customer UI.	Holding page is public; trade flow not qualified.

Module	Primary role	Qualification boundary
sdk	TypeScript and Python client helpers.	Interface support, not an independent guarantee.

B Database migration inventory

The baseline has twenty ordered PostgreSQL migrations. Their names indicate the evolving boundary; the table records the primary schema purpose, not proof of a deployed database. Applied migrations must be advanced through new forward files rather than rewritten.[Dip26e]

Version	Main schema responsibility
0001	Assets, accounts, integer balances, immutable journal transactions and legs, outbox, and initial deposit and withdrawal records.
0002	Deposit state transitions and canonical chain-event identity.
0003	Chain deployment registration, indexer checkpoints, and provider evidence.
0004	Reorganization handling and asset halt records.
0005	Withdrawal reservation constraints and customer identity uniqueness.
0006	Withdrawal authorizations and policy signatures.
0007	Versioned withdrawal limits and global halt controls.
0008	Withdrawal execution capability and authorization state.
0009	Withdrawal worker checkpoint state.
0010	Reverted attempts and verified replacement lineage.
0011	Continuous custody reconciliation evidence identity.
0012	Singleton engine leadership generation.
0013	Journal recovery sequence and watermark support.
0014	V3 omnibus custody and withdrawal authorization changes.
0015	Agent delegation events and notional buckets.
0016	Durable parent/child algorithm orders and events.
0017	Durable withdrawal broadcast queue.
0018	Relayer nonce reservation records.
0019	Prepared relayer transactions and submission outcomes.
0020	HTTP authentication challenges and account sessions.

C Launch evidence ledger

The M8 index is explicitly a template and evidence index, not an authorization to accept real funds. The following matrix restates its reviewed local evidence and missing independent gates. A benchmark result changes none of these statuses.[Dip26g]

Area	Available local evidence	Required external evidence still open
V3 vault	Foundry units, invariants, pause, replay, migration, and fixed-vector tests.	Deployed artifact identity, external settlement audit, remediation, and signed release revision.
Ledger and writer	PostgreSQL transaction, idempotency, policy, halt, and financial acknowledgment boundary tests.	Managed replication, encrypted backups, point-in-time restore, alerting, on-call ownership, and lost-machine rehearsal.
Deposits	Indexer finality, provider comparison, canonical event, reorganization, and halt tests.	Deployed V3 events, two independent hosted RPCs, manifest and runtime-code checks, and observed reconciliation.
Withdrawals	Reservation, authorization binding, ob-server, finality, and expiry-release tests.	Production signer or HSM/MPC, broadcaster, real chain transaction lineage, independent finality evidence, and adversarial rehearsal.

Area	Available local evidence	Required external evidence still open
Security	Source controls and local automated tests.	Independent API and infrastructure penetration test, funded bug bounty, and security sign-off.
Operations and legal	Written plans and evidence templates.	Capped adversarial testnet, incident and deploy rehearsals, customer disclosures, jurisdictions/assets review, surveillance, and written go/no-go approval.

D Benchmark evidence manifest

The source baseline is 940f3a48fe413fa83ff997fa3d6a770ac9488df8. The run IDs and SHA-256 digests below identify both campaigns. Each digest is split into two lines only for print layout.

Engine and legacy matching

20261008T221849Z-940f3a48fe41
Source diff: 5606bc8d2f07fcd9e41051c27dfbf467e
e0074bf43aceabb1baebb7cbd62e0ed

HTTP and sustained

20261008T225316Z-940f3a48fe41
Source diff: 811dc4740aaaa0a4d426b593b89d59
ba4983514af5db92063e27ef308251b391

Baseline evidence archive

dippi-benchmark-evidence-20261008.tar.gz
SHA-256: 1bc46eff4f8a36e0d2e1887ce39a7ea
4ad85ac71a451da16047b9f74166a2dff

The baseline archive includes the raw repetition directories, metadata, machine record, logs, and both source patches. The machine record and source patches are private companion evidence, not embedded in this public PDF.[\[Dip26a\]](#)

The follow-up source archive contains the exact Rust workspace files used for every follow-up suite. Its SHA-256 is d5b52c78bce4c760d4b2f43fd333ab39ebd99ef13ca95ef1c7faa41fe7d43f14. The follow-up run IDs are:

HTTP, one repetition

20261009T000155Z-d5b52c78bce4

HTTP, two repetitions

20261009T001108Z-d5b52c78bce4

Sustained, three repetitions

20261009T002116Z-d5b52c78bce4

Engine and matching, three repetitions

20261009T003751Z-d5b52c78bce4

Every run recorded the same source-archive digest.[\[Dip26b\]](#)

Follow-up evidence archive

dippi-rebenchmark-evidence-20261008.tar.gz
SHA-256: aea66214a43352c8a1e9c2fe767f3699
a1a59650ff04a997cec92a4d1147e68b

The archive includes all raw run directories, metadata, Criterion records, logs, and the exact source archive. Its remote and local digests matched before the second pod was stopped. The exploratory matching suite produced 30 identical named Criterion groups in each of three repetitions; those fixtures remain diagnostic rather than headline service measurements.

The tables identify distinct operation units. A Criterion iteration over 10,000 validated requests was divided by 10,000 to report an amortized cost; it is not a distribution of individual request latencies. Loopback HTTP p50 and p99 describe client-observed `send()` to complete-body time among successes. Sustained p50 and p99 end at response headers, and its throughput counts successful HTTP operations including public market-data reads. Failures remain in the attempt denominator. Cloud and local operating systems, CPU microarchitectures, and possible virtualization

contention limit direct comparison. A reader with authorized source access can rerun the commands in `paper/README.md` and regenerate both summaries with `paper/analyze_benchmarks.py` and `paper/analyze_rebenchmark.py`; the exact patched source is needed to reproduce the numbers.

References

- Dip26a. Dippi benchmark archive. Cpu benchmark evidence archive, October 2026. Private raw data, source patches, and machine record retained by the author. Archive SHA-256: `1bc46eff4f8a36e0d2e1887ce39a7ea4ad85ac71a451da16047b9f74166a2dff`. Summary: `paper/benchmark-summary.json`; reproduction script: `paper/analyze_benchmarks.py`.
- Dip26b. Dippi follow-up benchmark archive. Repaired-source cpu benchmark evidence, October 2026. Private raw data retained by the author. Evidence archive SHA-256: `aea66214a43352c8a1e9c2fe767f3699a1a59650ff04a997cec92a4d1147e68b`. Exact source archive SHA-256: `d5b52c78bce4c760d4b2f43fd333ab39ebd99ef13ca95ef1c7faa41fe7d43f14`. Summary: `paper/rebenchmark-summary.json`; reproduction script: `paper/analyze_rebenchmark.py`.
- Dip26c. Dippi source repository. Authoritative ledger and chain architecture decision records, 2026. Private repository, docs/adr/001-authoritative-ledger.md through docs/adr/008-initial-chain-and-assets.md, at source commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`.
- Dip26d. Dippi source repository. Consumer exchange engineering plan, 2026. Private repository, docs/consumer-exchange-engineering-plan.md, at source commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`.
- Dip26e. Dippi source repository. Exchange source tree at commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`, 2026. Private repository. Relevant paths: engine/, api/, ledger/, indexer/, contracts/, and benches/. Available to authorized reviewers.
- Dip26f. Dippi source repository. Free testnet staging operations, 2026. Private repository, docs/operations/free-testnet-staging.md, at source commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`.
- Dip26g. Dippi source repository. M8 launch evidence index, 2026. Private repository, docs/operations/m8-launch-evidence.md, at source commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`.
- Dip26h. Dippi source repository. Matcher to ledger commit boundary, 2026. Private repository, docs/architecture/matcher-ledger-commit.md, at source commit `940f3a48fe413fa83ff997fa3d6a770ac9488df8`.
- Som26. A. Somu. Vela: A high-performance verifiable spot exchange. SSRN, April 2026. Historical paper; its performance and deployment claims are not adopted as current findings here. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6579199.